Phase B — IP Candidate Extraction and Labelling

Phase B separates three ideas that are easy to confuse: (1) a regex match, (2) a syntactically valid IPv4 address, and (3) a source-labelled positive. Mismatches between them are kept and explained, not deleted. Input: IPs_500_SQLite_dataset.csv (hash verified in Phase A), column trimmed_text for context, IP_recoginzedby_regEx and label from the source.

1. Method

- Candidate extractor (loose regex): (?<!\d)\d+(?:\.\d+){3}(?!\d). It deliberately over-matches anything shaped like four dotted numbers.
- Independent validator: Python ipaddress.IPv4Address. It rejects octets above 255, leading zeros and non-numeric text, and records its error message.
- Context kept: the original string around each match (40 characters either side) is exported, together with checks for prefix/suffix contamination, longer dotted numbers (OIDs, versions), hostnames, and special ranges (0.0.0.0, loopback, private).
- Source comparison: the source records one IP per row. A candidate equal to that IP carries the source label; other candidates in the same text are marked unlabelled. If the source IP cannot be found by the regex, a row is still added with regex_match = False.

2. Demonstration samples (16 inputs)

#	Category	Candidate	Regex	Valid	Why
1	Genuine public IPv4	203.0.113.45	Yes	Yes	Valid but private/RFC1918 range
2	Genuine private IPv4	192.168.11.20	Yes	Yes	Valid but private/RFC1918 range
3	Version-like (AkamaiGHost)	9.9.4.1	Yes	Yes	Possible version-like context (heuristic: preceded by "/" or "version", or followed by "-digit")
4	Version-like (software)	1.18.0.2	Yes	Yes	Possible version-like context (heuristic: preceded by "/" or "version", or followed by "-digit")
5	Invalid octet >255	300.10.20.5	Yes	No	Invalid IPv4: Octet 300 (> 255) not permitted in '300.10.20.5'
5	Invalid octet >255	10.0.0.1	Yes	Yes	Valid but private/RFC1918 range
6	Invalid octet 999	192.168.1.999	Yes	No	Invalid IPv4: Octet 999 (> 255) not permitted in

					'192.168.1.999'
7	Leading zeros	010.001.002.003	Yes	No	Invalid IPv4: Leading zeros are not permitted in '010' in '010.001.002.003'
8	Suffix contamination (OID)	1.2.840.113635	Yes	No	Invalid IPv4: Octet 840 (> 255) not permitted in '1.2.840.113635'; Suffix contamination: followed by ".digit" so it is the head of a longer
8	Suffix contamination (OID)	100.6.2.12	Yes	Yes	Prefix contamination: preceded by "digit." so it is the tail of a longer dotted number
9	Prefix contamination (digit)	3608.80.24.1	Yes	No	Invalid IPv4: At most 3 characters permitted in '3608' in '3608.80.24.1'
10	Prefix contamination (longer dotted)	7.10.1.2	Yes	Yes	Suffix contamination: followed by ".digit" so it is the head of a longer dotted number (OID/version); Possible version-like context (heurist
11	Letters glued	10.1.2.3	Yes	Yes	Prefix contamination: glued to letters "abc"; Suffix contamination: glued to letters "xyz"; Valid but private/RFC1918 range
12	Embedded in hostname	23.204.143.86	Yes	Yes	Clean standalone valid IPv4
13	Embedded in URL	8.8.8.8	Yes	Yes	Possible version-like context (heuristic: preceded by "/" or "version", or followed by "-digit")
13	Embedded in URL	8.8.4.4	Yes	Yes	Clean standalone valid IPv4
14	Ambiguous placeholder	0.0.0.0	Yes	Yes	Valid but unspecified address 0.0.0.0 (placeholder)
14	Ambiguous placeholder	127.0.0.1	Yes	Yes	Valid but loopback
15	Ambiguous date/number	2020.04.02.1	Yes	No	Invalid IPv4: At most 3 characters permitted in '2020' in '2020.04.02.1'; Possible version-like context (heuristic: preceded by "/" or

					"vers
16	Three octets only (no match)	(none)	No	No	Not dotted-quad: regex does not fire

Takeaway: the regex alone accepts 300.10.20.5, 010.001.002.003, 3608.80.24.1 and 2020.04.02.1; only the validator rejects them. The validator alone accepts 9.9.4.1 and 1.18.0.2, which are valid IPv4 text yet look like version numbers; only context reveals this.

3. Regex vs valid IPv4 vs source label (510 source rows)

Outcome	Source label 1	Source label 0	No label
Regex found it, valid IPv4	390	116	5
Regex found it, not valid IPv4	1	68	0
Regex did not find it (malformed or shifted rows, absent)	0	0	10
Total	391	184	15

Candidate review CSV: 1,271 rows (590 source-selected, 681 other candidates found in the same text; 436 of the 681 are valid IPv4 but were never selected by the source, 245 are invalid).

4. Mismatches explained

- Valid but labelled 1 is not proof of an IP threat indicator: 137 positives sit in possible version-like contexts (e.g. AkamaiGHost/9.9.4.1-29027442), 63 are private RFC1918 addresses and 1 is loopback. The label reflects the source's own criterion, not syntactic validity.
- Valid but labelled 0: 116 cases. 42 are 0.0.0.0 placeholders; many others sit in version or OID text. Valid syntax and a negative label are consistent with each other here.
- Regex-matched but invalid: 69 selected rows. Mostly OID fragments such as 1.2.840.113 and 635.100.6.2 (octets above 255); the source regex truncated the longer token 1.2.840.113635.100.6.2.12. This is suffix contamination in the source itself.
- One label-1 row is invalid: 180.0.720.720 (octet 720). It is kept and flagged as a labelling error candidate for analyst review.
- Source IP not matched by the regex: 10 rows come from malformed or shifted CSV records (e.g. values like 0 or 00\x00") with no usable IP, and 5 of the 15 unlabelled rows are 0.0.0.0 placeholders.
- The dataset has 510 data rows against the documented 500; the extra 10 are the shifted rows above, so they should be reviewed before further training use.

5. Outputs

- candidate_review.csv — columns: source_row, context, candidate, regex_match, ipv4_valid, source_label, analyst_note.
- phase_b_demo_samples.csv — the 16 demonstration inputs and results.
- phase_b_extract.py and demo.py — reproducible code.

Limitations

The version-like and contamination flags are heuristics to guide analyst review, not definitive rules. Source labels exist only for the source-selected IP in each row.

6. Phase B Evidence Integrity Table

Before analysis, the evidence files in ~/CIP-107/originals were verified against SHA256_manifest.txt (sha256sum -c) and all six returned OK. Phase B outputs were then hashed with sha256sum and recorded in PhaseB_manifest.txt.

Filename	Description	SHA-256 Hash
candidate_review.csv	1,271-row candidate review (regex match, IPv4 validity, source label, analyst note)	309406ef396c15dd5f94f092990fc21f8e9df2689076a4d1b1e785e502fb36b7
phase_b_demo_samples.csv	16 demonstration samples (20 result rows)	e237ae6832ff06ec5e017c9c8b591c163727c5a0bd2cc2c0852c6df62d6a7b28
phase_b_extract.py	Regex candidate extractor and strict IPv4 validator	b7ce4d9c2724a8ad0b81c15f2b3c4b49bf976edaa21cd61bc6889af5a7120a7e
demo.py	Demonstration script	6980f30ce01fde73b9989a8d20675db7c47ec2c5c2f59f8c5361a57cc72c2545

Forensic Integrity Statement (Phase B)

The evidence files were read in place and never modified. All Phase B outputs were hashed with the Linux sha256sum utility. The results are reproducible by re-running phase_b_extract.py on originals/IPs_500_SQLite_dataset.csv. Hashing shows the outputs have not changed since they were produced; it does not validate the correctness of the source labels, which is why mismatches are documented in the analyst notes rather than deleted.